

OData service development with SAP Gateway – code-based service development – Part I

Introduction

In this blog I would like to show the basics of OData service development with SAP Gateway when using code based service implementation as it is shown in the upcoming SAP CodeJam events about OData service development with SAP Gateway. Though the recommended approach for OData service development in the most recent SAP NetWeaver releases is to use CDS views there are still a number of use cases where code based service implementation is a valid option.

1. Your SAP backend system is based on a SAP NetWeaver release 7.31 or earlier
2. Your system is based on SAP NetWeaver 7.40 or later but the business logic you want to re-use has been implemented in ABAP code
3. It is not possible to implement your business logic in CDS views

As an example we will use a simple OData model based that consist out of SalesOrders and SalesOrderItems with the option to navigate from a SalesOrder to the corresponding line items.

If you are more interested in using CDS views please have a look at my following post: [OData service development with SAP Gateway using CDS via Referenced Data Sources](#)

If you want to know how to annotate services using code based implemenation you can have a look at my following post:

<https://blogs.sap.com/2017/04/21/how-to-add-annotations-to-an-odata-service-using-code-based-implementation/>

Changes

- 28.06.2016 Fix for ABAP code: Replaced
`lv_osql_where_clause = io_tech_request_context->get_osql_where_clause( ).` through
`lv_osql_where_clause = io_tech_request_context->get_osql_where_clause_`**convert**`( ) .`
because otherwise the statement `GET SalesOrderSet?$filter=(Customer eq '100000000')` would not work because alpha conversion to `'0100000000'` would not take place.
- 24.11.2016 Moved ABAP source code into source code controls
- 21.04.2017 Added a link that explains how to add annotations to the service.
- 21.02.2018 Added a hint to un-check the Filter checkbox in SE24

Prerequisites

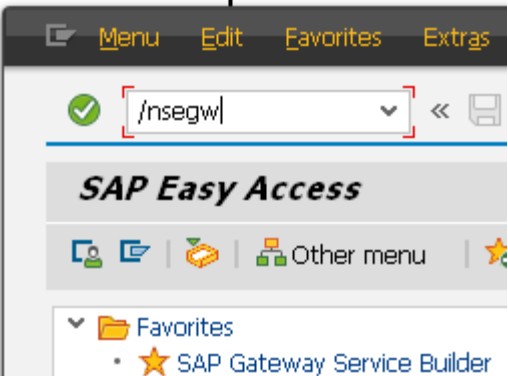
The examples shown in this blog are based on the latest SAP NetWeaver release 750so that we can compare the different implementation approaches

Creating a basic OData service

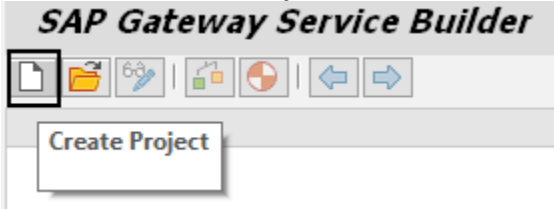
We will perform the following steps:

- Use the SAP Gateway Service Builder to create a new project
- Import a DDIC structures for SalesOrder to create the data model
- Generate, register and test the OData service using the Gateway Client
- Perform a first implementation of the GET_ENTITYSET method

Create a new Service Builder project



- Start the Gateway Service Builder by running transaction SEGW.
- Click on the Create Project button



- Provide the following information:

Project: ZE2E100_XX

Description: ZE2E100_XX

Service Package: \$TMP

Note: Replace XX with your session-id / group number.

Create Project

Project	ZE2E100_XX
Description	ZE2E100_XX Service

Attributes

Project Type	Service with SAP Annotations
Generation Strategy	Standard

Object Directory Entry

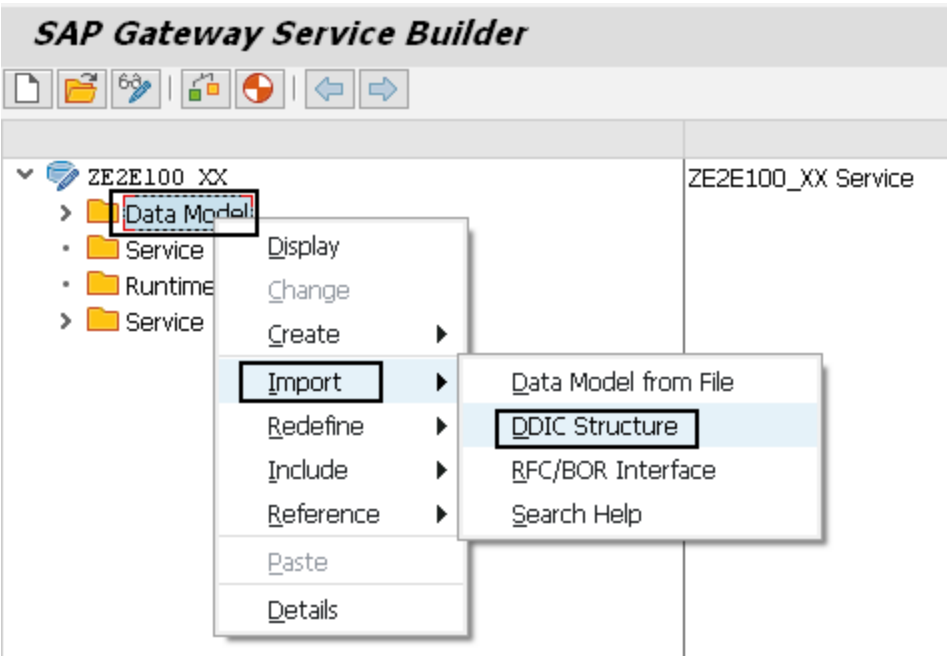
Package	\$TMP
Person Responsible	DEVELOPERXX

Local Object

- Press Continue
- Press **Save**

Import a DDIC structures for SalesOrder

- Right-click on Data Model and select Import -> DDIC Structure.



- In the first step of the wizard provide:
In the field Name, enter **SalesOrder**
In the field ABAP Structure, enter **SEPM_ISO**.

The screenshot shows the 'Wizard Step 1 of 3: Import from DDIC Structure' dialog. The 'Create an Entity Type or Complex Type' section has 'Name' set to 'SalesOrder' and 'Entity Type' selected. The 'Import from ABAP Structure' section has 'ABAP Structure' set to 'SEPM_ISO'. The 'Create Default Entity Set' checkbox is checked. At the bottom right are 'Next' and 'Cancel' buttons.

- Click **Next**.
- In the second step of the wizard provide:
Select the checkbox for SEPM_ISO.
Deselect the checkbox for **MANDT**,

The screenshot shows the 'Wizard Step 2 of 3: Import from DDIC Structure' dialog. The 'Select Parameter(s)' section shows a list of parameters. The 'MANDT' checkbox is selected, and the 'SEPM_ISO' checkbox is also selected. The table below shows the parameters and their details.

Data Source Parameter	Assign Structure	Description	Type	Length	Decimals	Import Search H...
☒ SEPM_ISO	☐		SEPM_ISO			☐
• ☒ MANDT	☐	Client	CLNT	3		☐
• ☒ SALESORDER	☐	Sales Order ID	CHAR	10		☐
• ☒ CREATIONDATETIME	☐	Created At	DEC	21	7	☐
• ☒ LASTCHANGEDDATETIME	☐	Changed At	DEC	21	7	☐
• ☒ ISCREATEDBYBUSINESSPARTNER	☐	Value	CHAR	1		☐
• ☒ ISLASTCHANGEDBYBUSINESSPARTNER	☐	Value	CHAR	1		☐
• ☒ CUSTOMER	☐	Business Partner ID	CHAR	10		☐
• ☒ TRANSACTIONCURRENCY	☐	Currency Code	CUKY	5		☐
• ☒ GROSSAMOUNTINTRANSACCURRENCY	☐	Gross Amount	CURR	15	2	☐
• ☒ NETAMOUNTINTRANSACTIONCURRENCY	☐	Net Amount	CURR	15	2	☐
• ☒ TAXAMOUNTINTRANSACTIONCURRENCY	☐	Tax Amount	CURR	15	2	☐
• ☒ SALESORDERLIFECYCLESTATUS	☐	Lifecycle Status	CHAR	1		☐
• ☒ SALESORDERBILLINGSTATUS	☐	Confirmation Status	CHAR	1		☐
• ☒ SALESORDERDELIVERYSTATUS	☐	Ordering Status	CHAR	1		☐
• ☒ SALESORDEROVERALLSTATUS	☐	Overall Status	CHAR	1		☐
• ☒ OPPORTUNITY	☐	Opportunity ID	CHAR	35		☐

At the bottom right are 'Back', 'Next', and 'Cancel' buttons.

- Click **Next**.
- In the third screen of the wizard
Select the checkbox **Is Key** for the field SALESORDER.
Click **Finish**.

Wizard Step 3 of 3: Import from DDIC Structure

Modify Entity Type

IsEn...	Complex/Entity Type	ABAP Name	Is Key	Type	Name	Label
☒	SalesOrder	SALESORDER	☒	CHAR	Salesorder	Sales Order ID
☒	SalesOrder	CREATIONDATETIME	☐	DEC	Creationdatetime	Created At
☒	SalesOrder	LASTCHANGEDDATETIME	☐	DEC	Lastchangeddatetime	Changed At
☒	SalesOrder	ISCREATEDBYBUSINESSP...	☒	CHAR	Iscreatedbybusinesspart...	Value
☒	SalesOrder	ISLASTCHANGEDBYBUSI...	☐	CHAR	Islastchangedbybusiness...	Value
☒	SalesOrder	CUSTOMER	☐	CHAR	Customer	Business Partner ID
☒	SalesOrder	TRANSACTIONCURRENCY	☐	CUKY	Transactioncurrency	Currency Code
☒	SalesOrder	GROSSAMOUNTINTRAN...	☐	CURR	Grossamountintransaccu...	Gross Amount
☒	SalesOrder	NETAMOUNTINTRANSA...	☐	CURR	Netamountintransaction...	Net Amount
☒	SalesOrder	TAXAMOUNTINTRANSA...	☐	CURR	Taxamountintransaction...	Tax Amount
☒	SalesOrder	SALESORDERLIFECYCLE...	☐	CHAR	Salesorderlifecyclestatu...	Lifecycle Status
☒	SalesOrder	SALESORDERBILLINGST...	☐	CHAR	Salesorderbillingstatus	Confirmation Status
☒	SalesOrder	SALESORDERDELIVERY...	☐	CHAR	Salesorderdeliverystatu...	Ordering Status
☒	SalesOrder	SALESORDEROVERALLS...	☐	CHAR	Salesorderoverallstatus	Overall Status
☒	SalesOrder	OPPORTUNITY	☐	CHAR	Opportunity	Opportunity ID

Back Finish Cancel

- Expand Folder Properties and examine the entity **SalesOrder** and the entity set **SalesOrderSet**. (For example the property names, the **Edm Core Type** values and the ABAP field names)

SAP Gateway Service Builder

Name	Key	Edm Core Type	Prec.	Scale	Max.	U.	Crea.	Upd.	Sort	Null.	Flt.	Label	L...	C...	ABAP Field Name	A.	Semantics
Salesorder	☒	Edm.String	0	0	10		☐	☐	☐	☐	☐	Sales Order ID	T		SALESORDER		
Creationdatetime	☐	Edm.DateTime	7	0	0		☐	☐	☐	☐	☐	Created At	T		CREATIONDATETIME		
Lastchangeddatetime	☐	Edm.DateTime	7	0	0		☐	☐	☐	☐	☐	Changed At	T		LASTCHANGEDDATETIME		
Iscreatedbybusinessp...	☐	Edm.Boolean	0	0	0		☐	☐	☐	☐	☐	Value	T		ISCREATEDBYBUSINESSPARTNER		
Islastchangedbybusin...	☐	Edm.Boolean	0	0	0		☐	☐	☐	☐	☐	Value	T		ISLASTCHANGEDBYBUSINESSPARTNER		
Customer	☐	Edm.String	0	0	10		☐	☐	☐	☐	☐	Business Partner ID	T		CUSTOMER		
Transactioncurrency	☐	Edm.String	0	0	5		☐	☐	☐	☐	☐	Currency Code	T		TRANSACTIONCURRENCY		
Grossamountintransac...	☐	Edm.Decimal	16	3	0		☐	☐	☐	☐	☐	Gross Amount	T		GROSSAMOUNTINTRANSACCURRENCY		
Netamountintransacti...	☐	Edm.Decimal	16	3	0		☐	☐	☐	☐	☐	Net Amount	T		NETAMOUNTINTRANSACTIONCURRENCY		
Taxamountintransacti...	☐	Edm.Decimal	16	3	0		☐	☐	☐	☐	☐	Tax Amount	T		TAXAMOUNTINTRANSACTIONCURRENCY		
Salesorderlifecyclestat...	☐	Edm.String	0	0	1		☐	☐	☐	☐	☐	Lifecycle Status	T		SALESORDERLIFECYCLESTATUS		
Salesorderbillingstatu...	☐	Edm.String	0	0	1		☐	☐	☐	☐	☐	Confirmation Status	T		SALESORDERBILLINGSTATUS		
Salesorderdeliverystat...	☐	Edm.String	0	0	1		☐	☐	☐	☐	☐	Ordering Status	T		SALESORDERDELIVERYSTATUS		
Salesorderoverallstatu...	☐	Edm.String	0	0	1		☐	☐	☐	☐	☐	Overall Status	T		SALESORDEROVERALLSTATUS		
Opportunity	☐	Edm.String	0	0	35		☐	☐	☐	☐	☐	Opportunity ID	T		OPPORTUNITY		

- Press Save
- Press the Check Project Consistency button.
- Verify that no errors were found.

- Press the **Generate Runtime Objects** button. **Note:** Using the Generate Runtime Objects button automatically saves the project.
- Leave all values as defaulted and press **Enter**.

Model and Service Definition

Model Provider Class

Class Name: ZCL_ZE2E100_XX_MPC_EXT

Base Class Name: ZCL_ZE2E100_XX_MPC

Data Provider Class

☒ Generate Classes

Class Name: ZCL_ZE2E100_XX_DPC_EXT

Base Class Name: ZCL_ZE2E100_XX_DPC

Service Registration

Technical Model Name: ZE2E100_XX_MDL

Model Version: 1

Technical Service Name: ZE2E100_XX_SRV

Service Version: 1

✓ ✗

- Choose **Local Object** in the Create Object Directory Entry popup.
- Verify that the generation was successful.

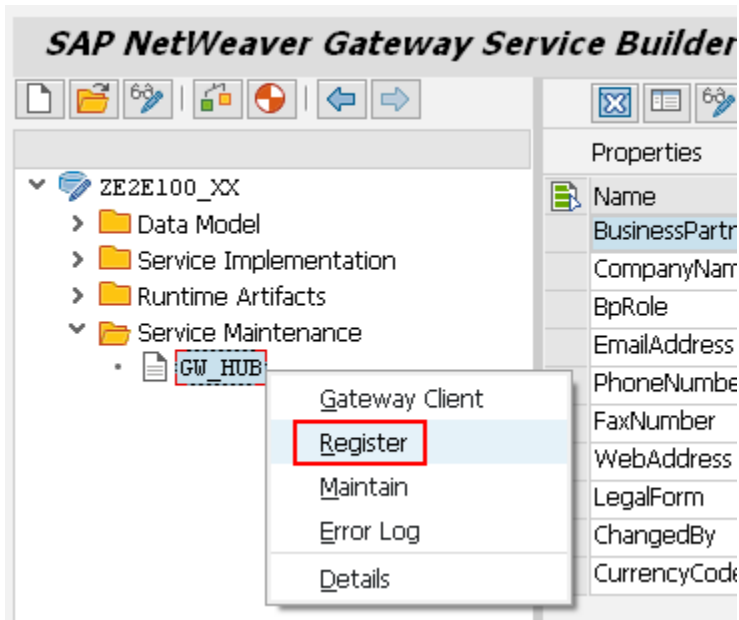
Messages

- Runtime objects for project 'ZE2E100_XX' were generated successfully
 - Model Provider Base Class 'ZCL_ZE2E100_XX_MPC' generated successfully
 - Model Provider Extension Class 'ZCL_ZE2E100_XX_MPC_EXT' generated successfully
 - Data provider base class ZCL_ZE2E100_XX_DPC generated successfully
 - Data provider implementation class ZCL_ZE2E100_XX_DPC_EXT generated successfully
 - Data provider base class ZCL_ZE2E100_XX_DPC implemented successfully
 - Service ZE2E100_XX_SRV was registered successfully
 - Model ZE2E100_XX_MDL was registered successfully

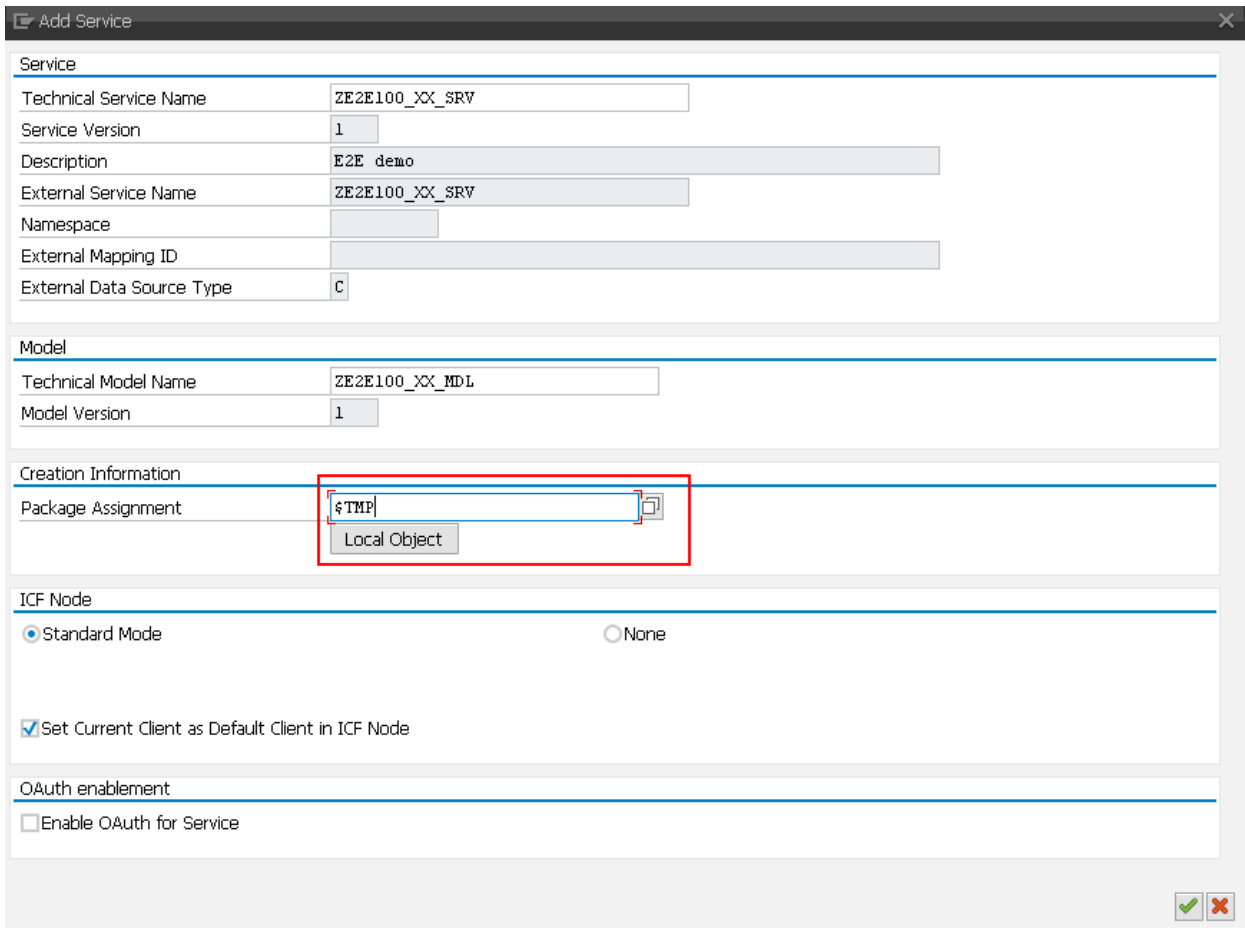
Now you have created an (empty) service implementation in the SAP backend.

Register and Test the OData service using the Gateway client

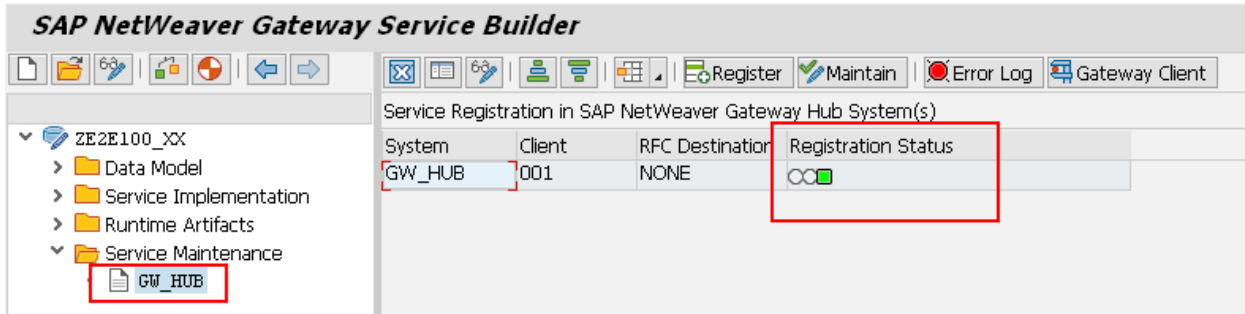
- Expand the node **Service Maintenance** and right-click on **GW_HUB** and select **Register**.



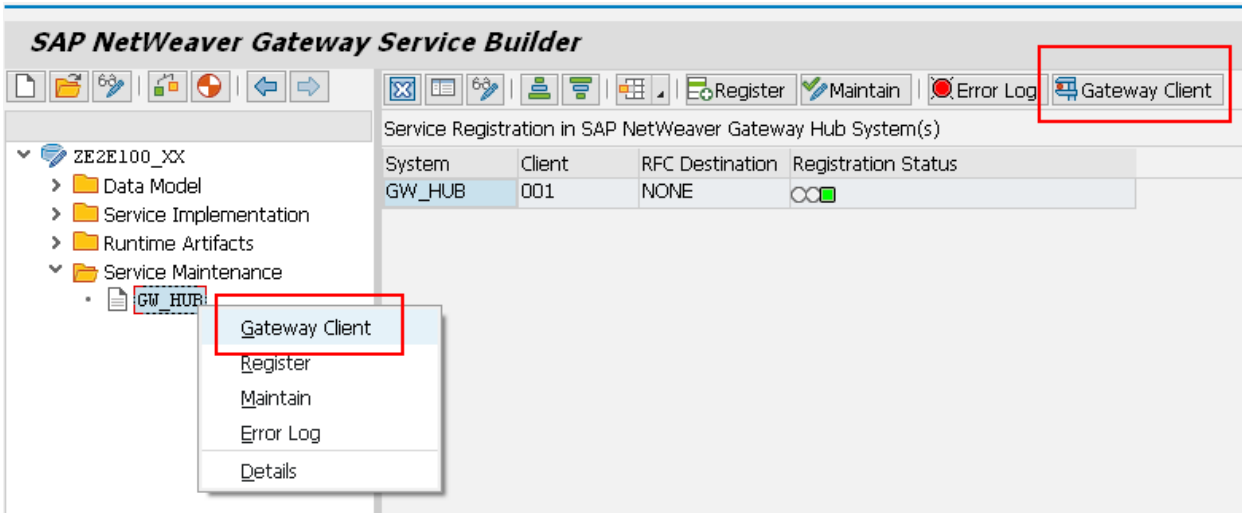
- In the Add Service popup leave all values as defaulted and press the Local Object button to assign the artifacts to the \$tmp package. Then press Enter to continue



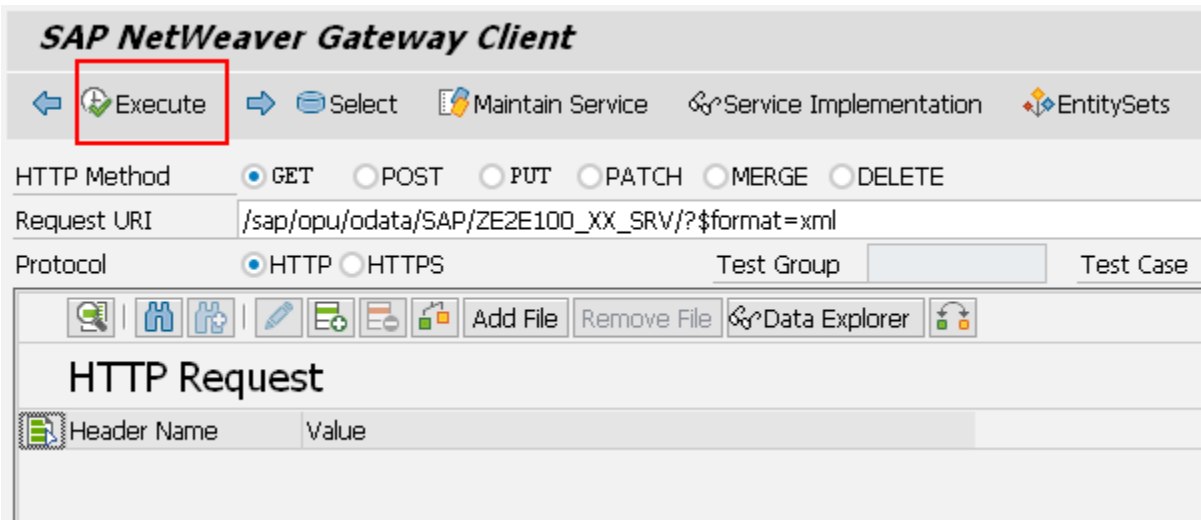
- Double-click on the **GW_HUB** entry under **Service Maintenance**. Verify that the registration status on the right-hand side is showing a green traffic light.



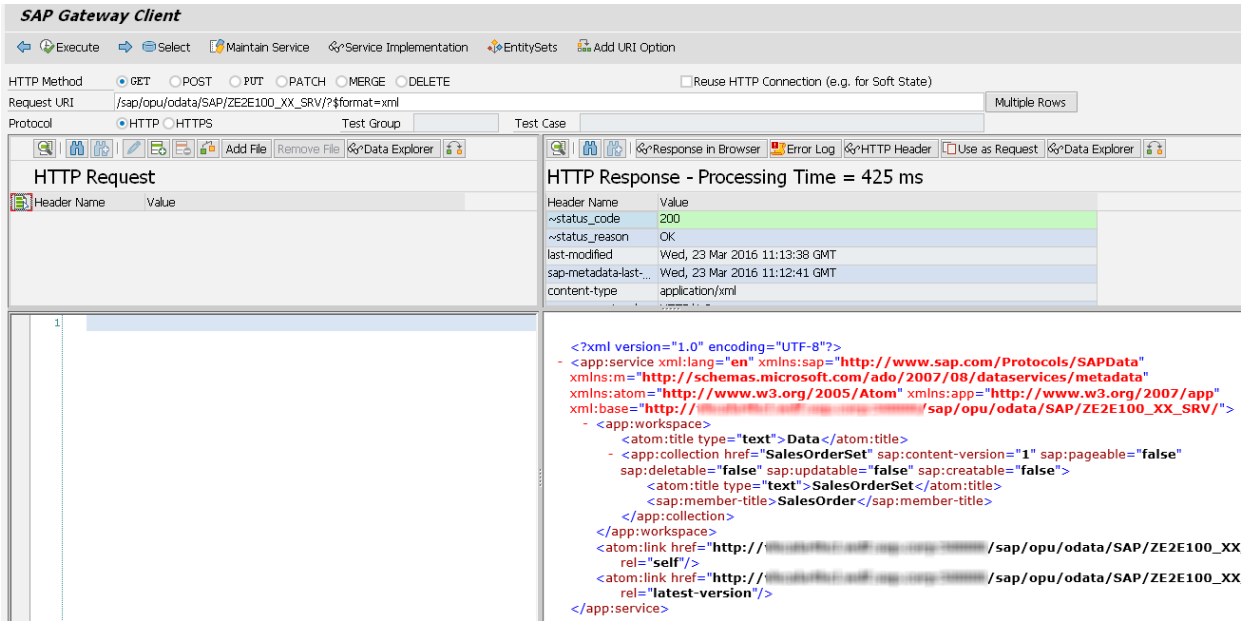
- In the navigation tree right-click on **GW_HUB** and select **SAP Gateway Client**. Alternatively click on the **SAP Gateway Client** button in the detail section.



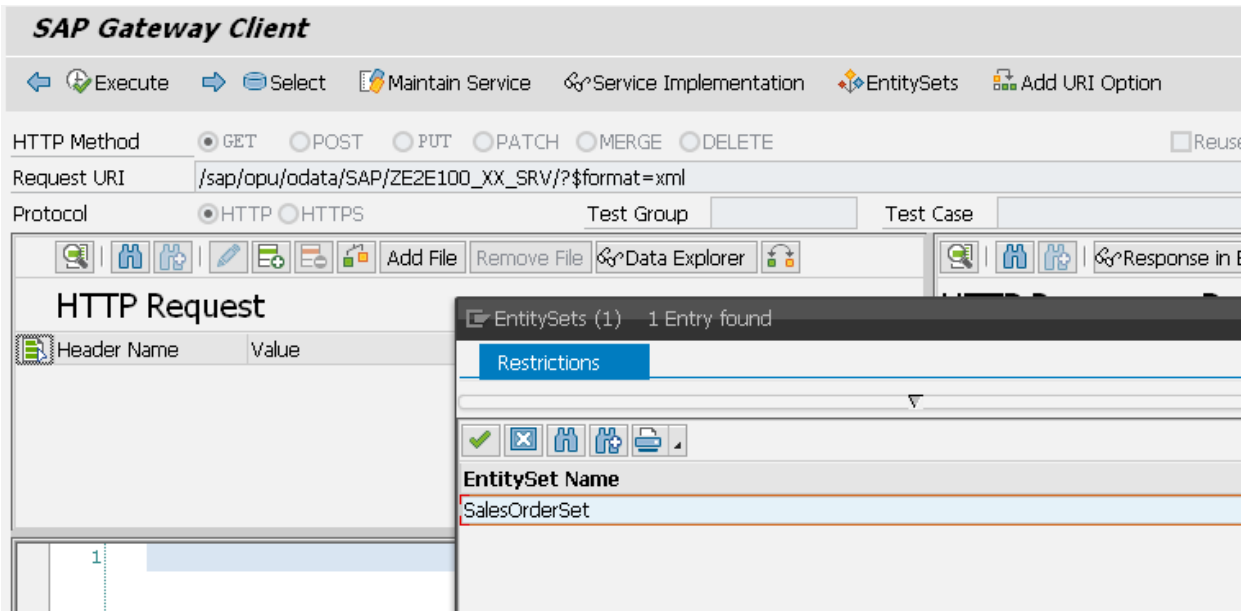
- Confirm the warning popup that warns you that you will be redirected to the selected system. Select **Yes**.
- This opens up the SAP Gateway Client in a new screen. The URI of the service document is already defaulted. Press the **Execute** button.



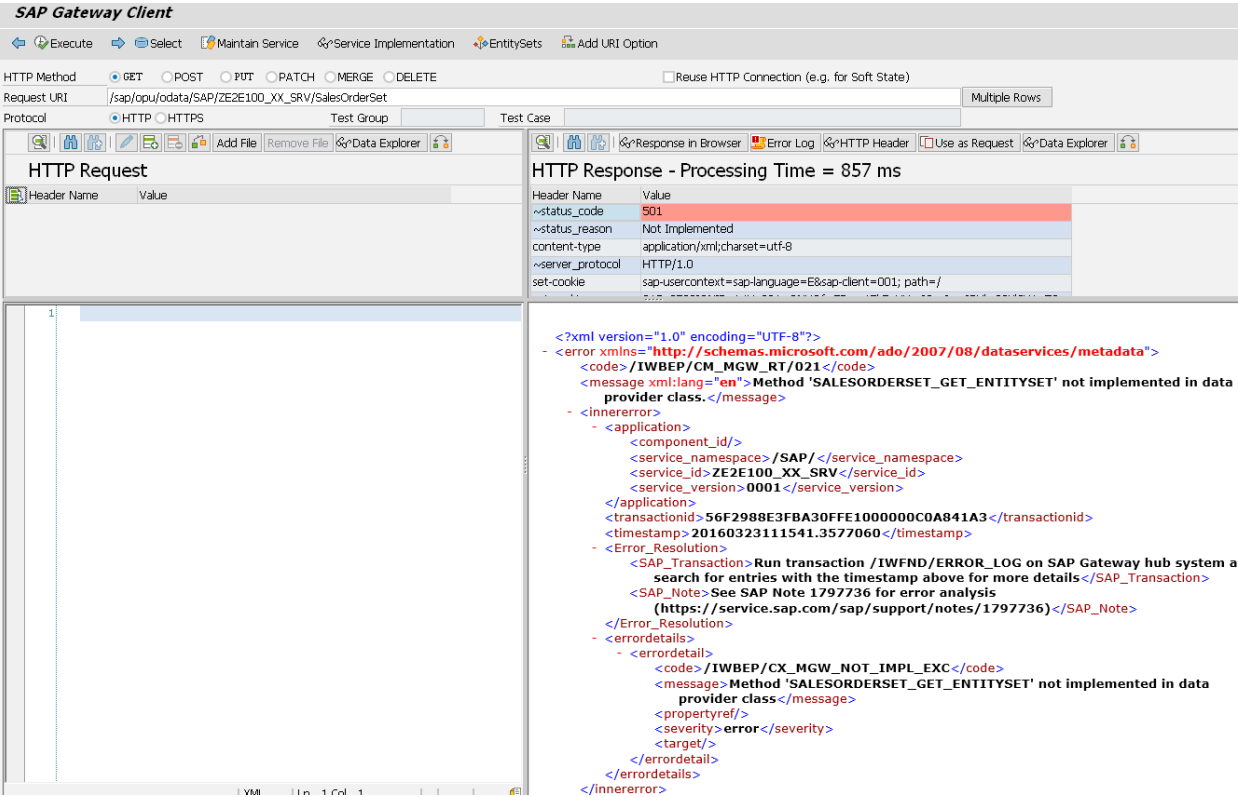
- As a result you get the service document of your OData service in the HTTP response window



- Press the Button “Entity Sets” and select **SalesOrderSet**.

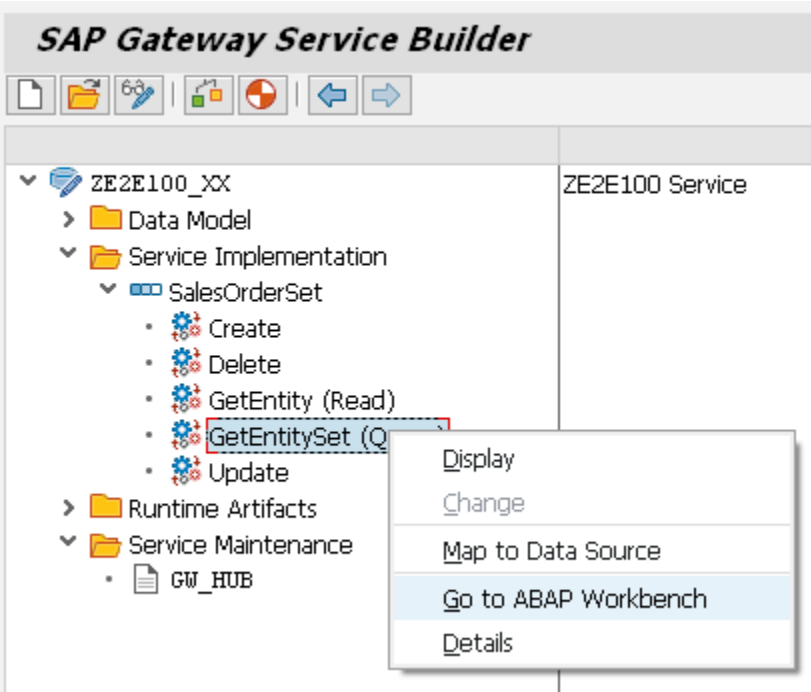


- After pressing **Execute** button which executes the following URI
/sap/opu/odata/SAP/ZE2E100_<XX>_SRV/SalesOrderSet
you see an error message that indicates that the method SALESORDERSET_GET_ENTITYSET has not yet been implemented.



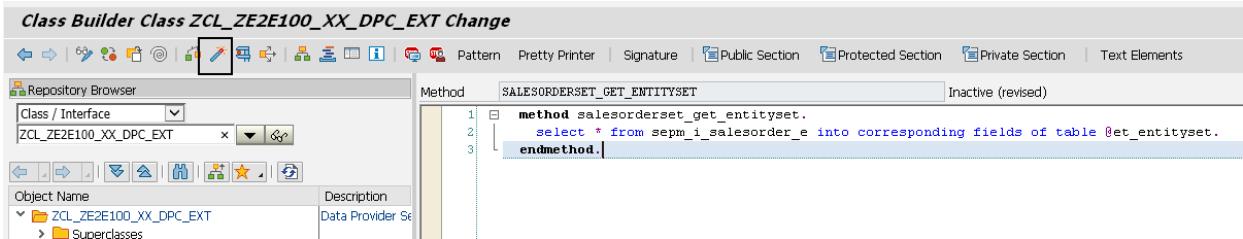
Provision the GET_ENTITYSET method of the service using ABAP code

- Open the Service Builder Project again (if you have closed it)
- Expand the node Service Implementation and expand **SalesOrderSet**. Now right-click on **GetEntitySet(Query)** and choose **Go to ABAP Workbench**.

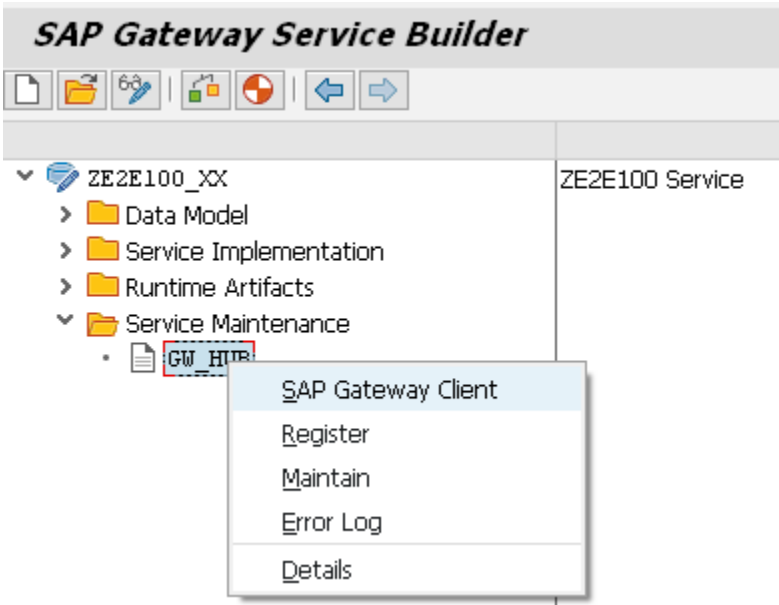


- Confirm the warning "Operation SALESORDERSET_GET_ENTITYSET has not yet been implemented"
- Switch to edit mode, scroll down to the method **SALESORDERSET_GET_ENTITYSET** and make sure to select it. Click on the **Redefine Method** button.

-
- `endmethod.`



- - **Info:** The replaced coding selects data from the CDS view `sepm_i_salesorder_e`. The results are filled into the corresponding fields of the return structure `et_entityset` of the `get-entityset` method. This works out of the box since the structure `SEPM_ISO` was used to create the entity type using DDIC import. This structure is the so called `sqlViewName` of the CDS view `sepm_i_salesorder_e`. The source code of the DDL source can be viewed using the report `RUTDDLSSHOW`.
- Click on **Activate**.
- Confirm the activation popup.
- Navigate back to the Service Builder main screen using the green back button multiple times.
- In the navigation tree right-click on **GW_HUB** and select **SAP Gateway Client**. Alternatively click on the **SAP Gateway Client** button in the detail section.

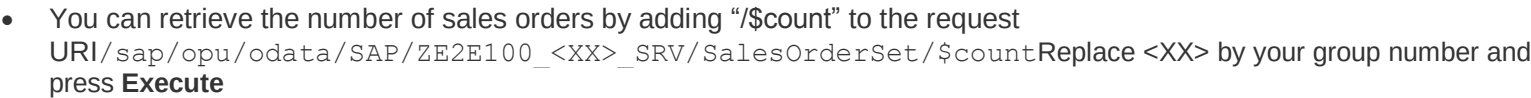


- This opens up the SAP Gateway Client in a new screen. The URI of the service document is already defaulted. Press the Button “Entity Sets” and select `SalesOrderSet`. The request URI field will contain the following value:

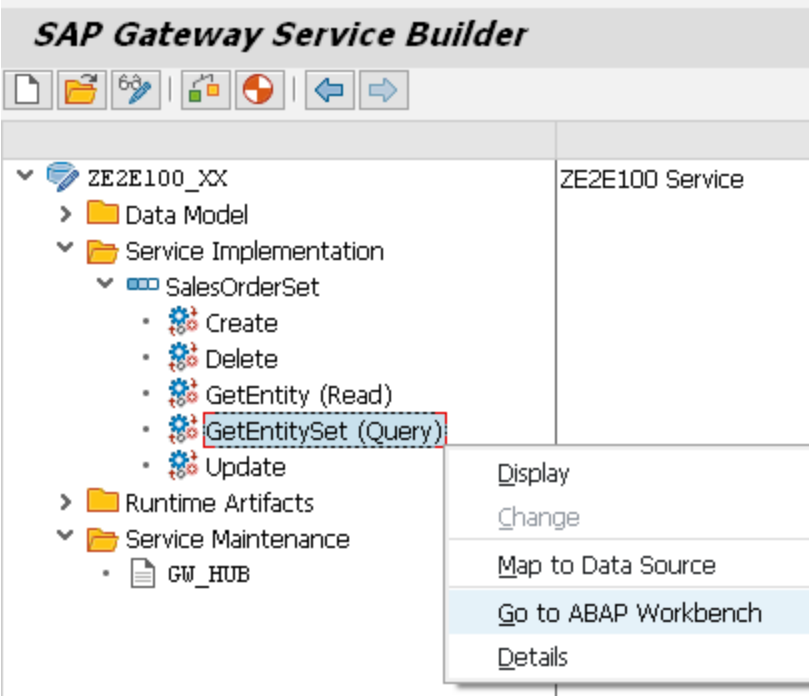
`/sap/opu/odata/SAP/ZE2E100_<XX>_SRV/SalesOrderSet`

(Replace `<XX>` by your group number.)

After pressing **Execute** button you see a list of sales orders.



- Open the Service Builder Project again
- Expand the node Service Implementation and SalesOrderSet and choose the entry GetEntitySet (Query). Now right-click on GetEntitySet (Query) and choose Go to ABAP Workbench.

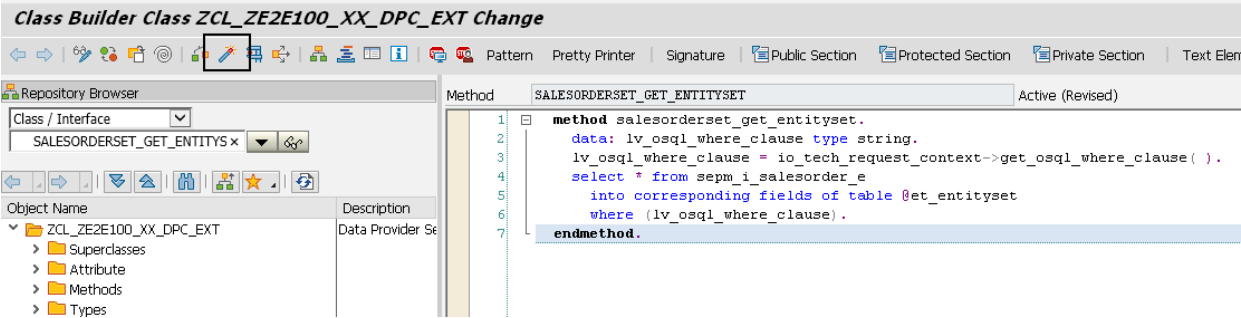


- This time the ABAP editor will open the method `salesorderset_get_entityset` immediately. Switch to the change mode and replace the code with the code shown below. The code retrieves the `$filter` statement as an `osql` statement for the where clause. The data is retrieved and inserted into the return table `et_entityset`.

```
method SALESORDERSET_GET_ENTITYSET.
```

```
data: lv_osql_where_clause type string.
lv_osql_where_clause = io_tech_request_context->get_osql_where_clause( ).
select * from sepm_i_salesorder_e
      into corresponding fields of table @et_entityset
      where (lv_osql_where_clause).

endmethod.
```



- Click on **Activate**.
- Navigate back to the Service Builder main screen using the back button
- In the navigation tree right-click on **GW_HUB** and select **SAP Gateway Client**. Alternatively click on the **SAP Gateway Client** button in the detail section.
- This opens up the SAP Gateway Client in a new screen. The URI of the service document is already defaulted.
- Replace the URI with the following:

`/sap/opu/odata/SAP/ZE2E100_<XX>_SRV/SalesOrderSet?$filter=Grossamountintransaccurrency ge 100000&$select=Salesorder,Grossamountintransaccurrency,Transactioncurrency&$format=json`

Replace `<XX>` by your group number and press **Execute**

This will deliver a list of four sales orders whose gross amount exceeds 100.000 Euro and will only show the sales order number and the gross amount.

-
- 500000005
- 500000030
- 500000034
- 500000045
-

```
{
  "d" : {
    "results" : [
      {
        "__metadata" : {
        },
        "Salesorder" : "500000005",
        "Transactioncurrency" : "EUR",
        "Grossamountintransaccurrency" : "101299.22"
      },
      {
        "__metadata" : {
        },
        "Salesorder" : "500000030",
        "Transactioncurrency" : "EUR",
        "Grossamountintransaccurrency" : "101299.22"
      },
      {
        "__metadata" : {
        },
        "Salesorder" : "500000034",
        "Transactioncurrency" : "EUR",
        "Grossamountintransaccurrency" : "107100.00"
      },
      {

```

Add support for additional query options to your code

- Open the Service Builder Project again
- Expand the node Service Implementation and right-click on **SalesOrderSet** and choose the entry **GetEntitySet (Query)**. Now right-click on **GetEntitySet (Query)** and choose **Go to ABAP Workbench**.

- This time the ABAP editor will open the method `salesorderset_get_entityset` immediately.
- Switch to the edit mode.
Replace the code with the code shown below.
The code retrieves the values for `$top` and `$skip` and the `$filter` statement as an osql statement for the where clause. The data is retrieved and in the end it is checked whether `$inlinecount` is used (which is a default for SAPUI5)

```
method SALESORDERSET_GET_ENTITYSET.

  data: lv_osql_where_clause type string,
        lv_top              type i,
        lv_skip             type i,
        lv_max_index        type i,
        n                   type i.

  *- get number of records requested
  lv_top = io_tech_request_context->get_top( ).
  *- get number of lines that should be skipped
  lv_skip = io_tech_request_context->get_skip( ).
  *- value for maxrows must only be calculated if the request also contains a $top
  if lv_top is not initial.
    lv_max_index = lv_top + lv_skip.
  endif.
  lv_osql_where_clause = io_tech_request_context->get_osql_where_clause( ).

  select * from sepm_i_salesorder_e
    into corresponding fields of table @et_entityset
    up to @lv_max_index rows
    where (lv_osql_where_clause).
  *- skipping entries specified by $skip
  if lv_skip is not initial.
    delete et_entityset to lv_skip.
  endif.
  *- Inlinecount - get the total numbers of entries that fit to the where clause
  if io_tech_request_context->has_inlinecount( ) = abap_true.
    select count(*) from sepm_i_salesorder_e where (lv_osql_where_clause) .
    es_response_context-inlinecount = sy-dbcnt.
  else.
    clear es_response_context-inlinecount.
  endif.

endmethod.
```

- Click on **Activate**.

- Navigate back to the Service Builder main screen using the back button multiple times.
- In the navigation tree right-click on **GW_HUB** and select **SAP Gateway Client**.Alternatively click on the **SAP Gateway Client** button in the detail section.
- This opens up the SAP Gateway Client in a new screen. The URI of the service document is already defaulted.
- Replace the URI with the following:
`/sap/opu/odata/SAP/ZE2E100_<XX>_SRV/SalesOrderSet?$filter=Grossamountintransaccurrency ge 100000&$select=Salesorder,Grossamountintransaccurrency,Transactioncurrency&$top=2&$skip=1&$inlinecount=allpages&$format=json` Replace `<XX>` by your group number and press **Execute**
This will deliver a list of only 2 sales orders whose gross amount exceeds 100.000 EUR. The first sales order `500000005` is skipped and only the first 2 of the rest of the list (highlighted in bold) are shown.
The inlinecount however shows that in total 40 sales orders would fit to the filter criteria.

500000005

500000030 <—

500000034 <—

500000045Please note: \$skip, \$top and \$inlinecount are used for client side paging. SAPUI5 uses this to calculate how many pages one has to navigate through.

```
1  {
2  □  "d" : {
3    |    "__count" : "40",
4    |    "results" : [
5      |    {
6  +    |      "__metadata" : {
10   |    },
11  □  |    "Salesorder" : "500000030",
12   |    "Transactioncurrency" : "EUR",
13   |    "Grossamountintransaccurrency" : "101299.22"
14   |    },
15   |    {
16  +    |      "__metadata" : {
20   |    },
21  □  |    "Salesorder" : "500000034",
22   |    "Transactioncurrency" : "EUR",
23   |    "Grossamountintransaccurrency" : "107100.00"
24   |    }
25   |  ]
26   }
27 }
```

What is now left is the implementation of the GET_ENTITY method of the entity set SalesOrderSet and the modelling and implementation of the navigation between sales order header and the line items.